

Precise Zero-Shot Dense Retrieval without Relevance Labels

Luyu Gao*, Xueguang Ma*, Jimmy Lin, Jamie Callan Carnegie Mellon University • University of Waterloo

ACL, 2022/2023 • [arXiv:2212.10496](https://arxiv.org/abs/2212.10496)

<https://arxiv.org/abs/2212.10496>

Abstract

While dense retrieval has been shown effective and efficient across tasks and languages, it remains difficult to create effective fully zero-shot dense retrieval systems when no relevance label is available. In this paper, we recognize the difficulty of zero-shot learning and encoding relevance. Instead, we propose to pivot through Hypothetical Document Embeddings (HyDE). Given a query, HyDE first zero-shot instructs an instruction-following language model (e.g. InstructGPT) to generate a hypothetical document. The document captures relevance patterns but is unreal and may contain false details. Then, an unsupervised contrastively learned encoder (e.g. Contriever) encodes the document into an embedding vector. This vector identifies a neighborhood in the corpus embedding space, where similar real documents are retrieved based on vector similarity. Our experiments show that HyDE significantly outperforms the state-of-the-art unsupervised dense retriever Contriever and shows strong performance comparable to fine-tuned retrievers across various tasks (web search, QA, fact verification) and languages (Swahili, Korean, Japanese).

1. Introduction

Dense retrieval, the method of retrieving documents using semantic embedding similarities, has been shown successful across tasks like web search, question answering, and fact verification. A variety of methods such as negative mining, distillation, and task-specific pre-training have been proposed to improve the effectiveness of supervised dense retrieval models. On the other hand, zero-shot dense retrieval still remains difficult. Many recent works consider the alternative transfer learning setup, where the dense retrievers are trained on a high-resource dataset and then evaluated on queries from new tasks. The MS-MARCO collection, a massive judged dataset, is arguably the most commonly used. However, as argued by prior work, in practice the existence of such a large dataset cannot always be assumed.

In this paper, we aim to build effective fully zero-shot dense retrieval systems that require no relevance supervision, work out-of-box and generalize across tasks. As supervision is not available, we start by examining self-supervised representation learning methods. We propose to pivot through Hypothetical Document Embeddings (HyDE), and decompose dense retrieval into two tasks: a generative task performed by an instruction-following language model and a document-document similarity task performed by a contrastive encoder.

2. Methodology

2.1 Problem Formulation

Dense retrieval models similarity between query and document with inner product similarity. Given a query q and document d , it uses two encoder functions enc_q and enc_d to map them into d -dimension vectors v_q and v_d , whose inner product is used as similarity measurement: $\text{sim}(q, d) = v_q \cdot v_d$. For zero-shot retrieval, we consider L query sets and their corresponding search corpora without access to any relevance judgment.

2.2 HyDE Algorithm

HyDE circumvents the learning problem by performing search in document-only embedding space that captures document-document similarity. This can be easily learned using unsupervised contrastive learning. HyDE proceeds in two steps:

Step 1 — Hypothetical Document Generation: Given a query q , HyDE instructs an instruction-following LM to generate a hypothetical document that answers the query. The instruction is simply "write a paragraph that answers the question". The generated document is not real and may contain factual errors, but it captures the relevance pattern — it is like a relevant document.

Step 2 — Dense Bottleneck Encoding: The hypothetical document is encoded using an unsupervised contrastive encoder (e.g., Contriever) into an embedding vector. This vector identifies a neighborhood in the corpus embedding space. The encoder's dense bottleneck serves as a lossy compressor, filtering out the hallucinated details while preserving semantic relevance patterns. The most similar real documents are then retrieved based on this vector.

Formally: $E[v_q] = E[f(g(q, \text{INST}))]$ where g is the generative LM and f is the contrastive encoder. In practice, HyDE estimates this by sampling N hypothetical documents and averaging their embeddings: $v_{\text{hat}_q} = (1/N) * \sum_k f(d_{\text{hat}_k})$. The original query can also be included in the average.

3. Experimental Results

3.1 Web Search (TREC DL19/DL20)

System	DL19 MAP	DL19 NDCG@10	DL19 Recall@1k	DL20 MAP	DL20 NDCG@10
BM25	30.1	50.6	75.0	28.6	48.0
Contriever (no label)	24.0	44.5	74.6	24.0	42.1
HyDE (proposed)	41.8	61.3	88.0	38.2	57.9
ContrieverFT (supervised)	41.7	62.1	83.6	43.6	63.2

HyDE significantly outperforms unsupervised Contriever and shows performance comparable to supervised ContrieverFT, despite requiring zero relevance labels. HyDE outperforms BM25 by large margins on both MAP and NDCG@10.

3.2 Low-Resource Retrieval (BEIR Benchmark)

Dataset	BM25 nDCG@10	Contriever nDCG@10	HyDE nDCG@10	ContrieverFT nDCG@10
Scifact	67.9	64.9	69.1	67.7

Arguana	39.7	37.9	46.6	44.6
TREC-Covid	59.5	27.3	59.3	59.6
FiQA	23.6	24.5	27.3	32.9
DBPedia	31.8	29.2	36.8	41.3

HyDE demonstrates strong performance across diverse low-resource tasks, generally outperforming both unsupervised Contriever and supervised DPR/ANCE models, despite being fully unsupervised.

3.3 Effect of Generative Model Size

Generative Model	Encoder	DL19 NDCG@10	DL20 NDCG@10
Flan-T5 (11B)	Contriever	48.9	52.9
Cohere (52B)	Contriever	53.8	53.8
InstructGPT (175B)	Contriever	61.3	57.9
Flan-T5 (11B)	ContrieverFT	60.2	62.1
InstructGPT (175B)	ContrieverFT	67.4	63.5

Larger generative models consistently produce better hypothetical documents, leading to improved retrieval performance. This suggests that as LLMs improve, HyDE quality will scale accordingly.

4. Key Insights for RAG Practitioners

1. The query-document gap: Short user queries are semantically distant from long document passages. HyDE bridges this by generating a "hypothetical passage" that looks like a real answer, making the embedding comparison document-document rather than query-document.
2. Hallucination is acceptable at indexing time: The hypothetical document may be factually wrong. The dense encoder's bottleneck discards factual specifics and retains only semantic patterns — the wrong facts literally get compressed away.
3. Production implementation: In production RAG systems, you can implement HyDE in two ways: (a) At query time — call LLM to generate hypothetical answer, embed it, search. Adds ~500ms latency. (b) At indexing time — generate hypothetical questions per chunk (reverse HyDE), embed them, search at query time with real query. Zero latency overhead.
4. Limitations: HyDE can mislead retrieval if the LLM generates a hypothetical document with wrong domain terminology. For highly specialized domains (medical, legal), ensure the LLM has domain knowledge or use domain-specific instruction prompts. For fact-bound exact queries (product codes, invoice numbers), BM25/sparse retrieval is more reliable than HyDE.

5. Citation

Gao, L., Ma, X., Lin, J., & Callan, J. (2022). Precise Zero-Shot Dense Retrieval without Relevance Labels. Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL

2023). [arXiv:2212.10496](#).